

Backend Engineer Technical Assignment

Overview

Thanks for taking the time to do this. The goal is to give us a realistic picture of how you design and build a backend service, and to give you something concrete to talk us through later.

You will build a small multi-tenant asset service that uses both a relational and a document store. The dataset is generic (no domain knowledge required), and the brief is intentionally small. We are not looking for a finished product. We are looking for the design decisions you make and how you defend them.

AI tools

AI tools (Cursor, Copilot, Claude, GPT, anything you normally use) are allowed and encouraged. We use them too.

The problem

Build a small multi-tenant backend service for an asset management product. The service hosts several independent organizations (tenants). Each tenant manages its own assets. The asset shape can vary per tenant.

The service should support:

- Onboarding tenants and managing users inside each tenant.
- Authenticated, tenant-scoped access. A request from one tenant must never see another tenant's data.
- CRUD on assets, scoped to the calling tenant.
- A cross-store report endpoint that combines tenant metadata (Postgres) with asset data (Mongo).

The core design question we want you to answer with this submission: **what belongs in the relational store, what belongs in the document store, and why?**

The data

Two seed files are provided alongside this brief:

- **tenants.seed.sql**: three tenants, each with a small set of users and a single role per user.
- **assets.seed.json**: around 100 assets per tenant. All tenants share a core shape (id, tenant_id, name, type, status, lat, lng, installed_at), but each tenant adds a different set of extra fields, so the document schemas are not identical. That variability is part of what makes the document store a sensible fit for that side of the data.

What to build

Stack

- Node.js / Express, TypeScript.
- PostgreSQL for tenants, users, and roles (relational).
- MongoDB for asset documents (document).
- A simple bearer-token auth scheme is fine. No OAuth or SSO required.

Capabilities

- Tenant onboarding and user management inside a tenant. API design (endpoints, request and response shapes, pagination, error contracts) is up to you and is part of what we evaluate.
- Tenant-scoped CRUD on assets. Filtering should at a minimum cover type and status.
- At least one cross-store report endpoint that joins tenant metadata from Postgres with asset data from Mongo. Pick a useful one (for example, asset counts grouped by status for the caller's tenant, or a recent-activity summary).
- Indexes in both stores: pick what you need.
- Seed Postgres from the SQL file and Mongo from the JSON file on startup or via a seed script.

Tenant isolation

Tenant isolation is the single most important correctness concern. Every authenticated request must carry a tenant context, and every query must be scoped by it. The seam in the code where this is enforced should be unambiguous.

Optional

A caching layer over one or more read-heavy endpoints. The interesting parts are what to cache, how it is invalidated on writes, and how keys are scoped per tenant. In-process or out-of-process is up to you.

Not required

To save you time, the following are explicitly out of scope:

- Any frontend or UI.
- OAuth, SSO, or any external identity provider.
- Deployment. Local-run is fine.
- Exhaustive tests. A small handful of meaningful ones (especially around tenant isolation) is more useful than full coverage.
- Production observability beyond basic logs.
- Horizontal scaling, sharding, or replica configuration.

Deliverables

- A public Git repo with the running service.

What we look for

- A defensible split between Postgres and Mongo.
- Sensible API design (resource shapes, query design, error handling, pagination).
- Tenant isolation that holds up to scrutiny.
- Indexes that map to the queries you actually expose.
- Code that reads well: layered, separated, easy to extend.

Next steps

We will review your submission and get back to you within about a week.

Have fun with it.